APACHE HADOOP MAPREDUCE

Assignment 1: Data Engineering

NAME	CHANDANA K S
USN	4GW23CI008
SEMESTER	7th

Sports Score Analysis Using Python mrjob and Apache Hadoop MapReduce

SPORTS SCORE	HDFS	MAP	REDUCE	AVERAGE / PLAYER
Input	Storage	Player + score	Sum + count	Final result

1. Aim

To write a MapReduce program in Python using the mrjob library that reads sports score data stored in HDFS and calculates the average score for each player.

2. Problem Statement

A sports organization collects the scores made by players in different matches. Each record contains the player name and the score obtained by that player. As the amount of match data increases, it can be stored and processed efficiently using HDFS and Hadoop MapReduce.

The task is to write a MapReduce program using Python's mrjob framework that reads the sports score data, extracts the player name and score, and calculates the average score for each player. In the Map phase, each input record is split into the player name and score, and the program outputs the pair (player, score). Hadoop then groups all scores belonging to the same player. In the Reduce phase, the scores for each player are added and divided by the number of scores to calculate the average score.

3. Technologies Used

Technology	Purpose
Ubuntu 24.04 LTS	Operating environment for Hadoop
OpenJDK 17	Java runtime required by Hadoop
Apache Hadoop 3.5.0	Distributed data processing framework
HDFS	Stores the sports score input data
YARN	Manages resources and supports MapReduce execution
Python 3.12	Programming language
mrjob	Python framework for Hadoop MapReduce jobs

4. Hadoop Configuration

The Hadoop environment is configured in pseudo-distributed mode. The configuration supports HDFS storage, MapReduce execution through YARN, and the shuffle service required for grouping Mapper output before the Reducer.

core-site.xml: defines the default filesystem used by Hadoop.

```
<property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
</property>
```

hdfs-site.xml: configures HDFS storage. A replication factor of 1 is suitable for a single-machine setup.

```
<property>
    <name>dfs.replication</name>
    <value>1</value>
</property>
```

mapred-site.xml: configures MapReduce to use YARN.

```
<property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
</property>
```

yarn-site.xml: enables the MapReduce shuffle service required for grouping Mapper output before the Reducer.

```
<property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
</property>
```

The terminal evidence used for this report is shown in the execution section below.

5. HDFS Input and Hadoop Services

The sports score file is stored in HDFS and is used as the input to the MapReduce job. The terminal evidence shows the local score records, HDFS directory creation, file upload, HDFS listing, and the file contents.

```
hdfs dfs -mkdir -p /user/$USER/sports
hdfs dfs -put scores.txt /user/$USER/sports/
hdfs dfs -ls /user/$USER/sports/
hdfs dfs -cat /user/$USER/sports/scores.txt
```

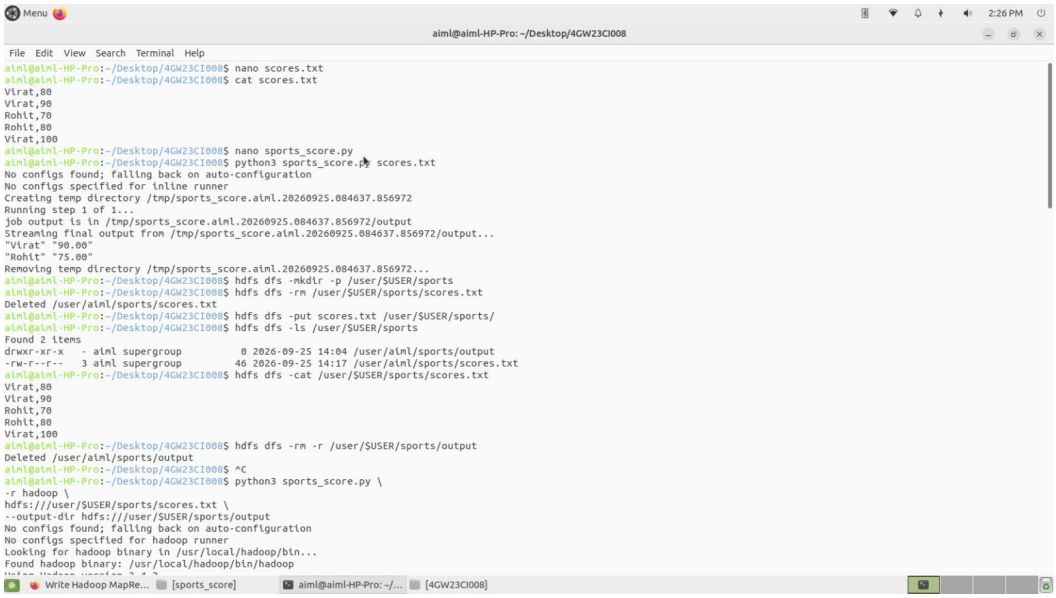


Figure 1. Terminal evidence showing sports score input and HDFS storage.

The HDFS input contains the five records: Virat,80; Virat,90; Rohit,70; Rohit,80; and Virat,100.

6. Python mrjob Implementation

File name: sports_score.py

```
from mrjob.job import MRJob


class SportsScore(MRJob):

    def mapper(self, _, line):
        player, score = line.split(",")

        yield player, float(score)

    def reducer(self, player, scores):
        scores = list(scores)

        total = sum(scores)
        count = len(scores)

        average = total / count

        yield player, f"{average:.2f}"


if __name__ == "__main__":
    SportsScore.run()
```

mapper(): The mapper reads each input line and splits it using the comma separator. For example, Virat,80 becomes Player = Virat and Score = 80. The mapper outputs Virat → 80.

reducer(): The reducer receives one player name and all scores belonging to that player. It calculates Average Score = Sum of Scores / Number of Scores.

For Virat: 80 + 90 + 100 = 270; Count = 3; Average = 270 / 3 = 90.00. For Rohit: 70 + 80 = 150; Count = 2; Average = 150 / 2 = 75.00.

run(): The run() function starts the MapReduce job. The option -r hadoop makes mrjob submit and execute the job using the Hadoop cluster through Hadoop Streaming.

7. Execution of the MapReduce Job

The Hadoop runner is invoked with the HDFS input path and an HDFS output directory.

```
python3 sports_score.py \
-r hadoop \
hdfs:///user/$USER/sports/scores.txt \
--output-dir hdfs:///user/$USER/sports/output
```

The terminal evidence shows Hadoop Streaming being used, the job connecting to the ResourceManager, Map and Reduce tasks reaching 100%, and the job completing successfully.

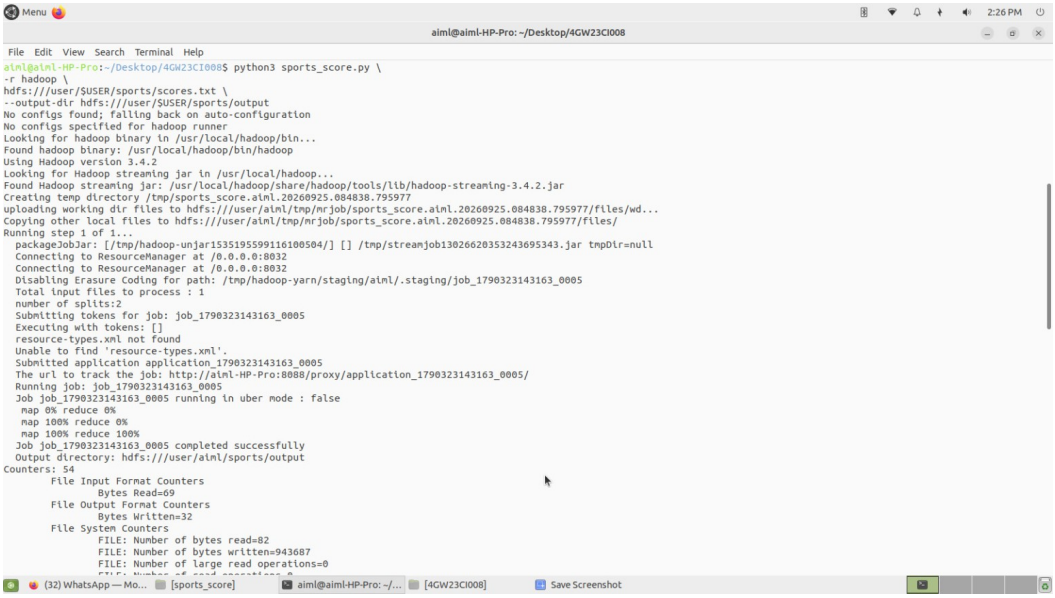


Figure 2. Successful Hadoop MapReduce execution with Map and Reduce tasks completed.

8. Execution Result and Hadoop Counters

The completed Hadoop job reports the generated HDFS output directory and MapReduce execution counters. The supplied terminal evidence shows five Map input records, five Map output records, two Reduce input groups, and two Reduce output records.

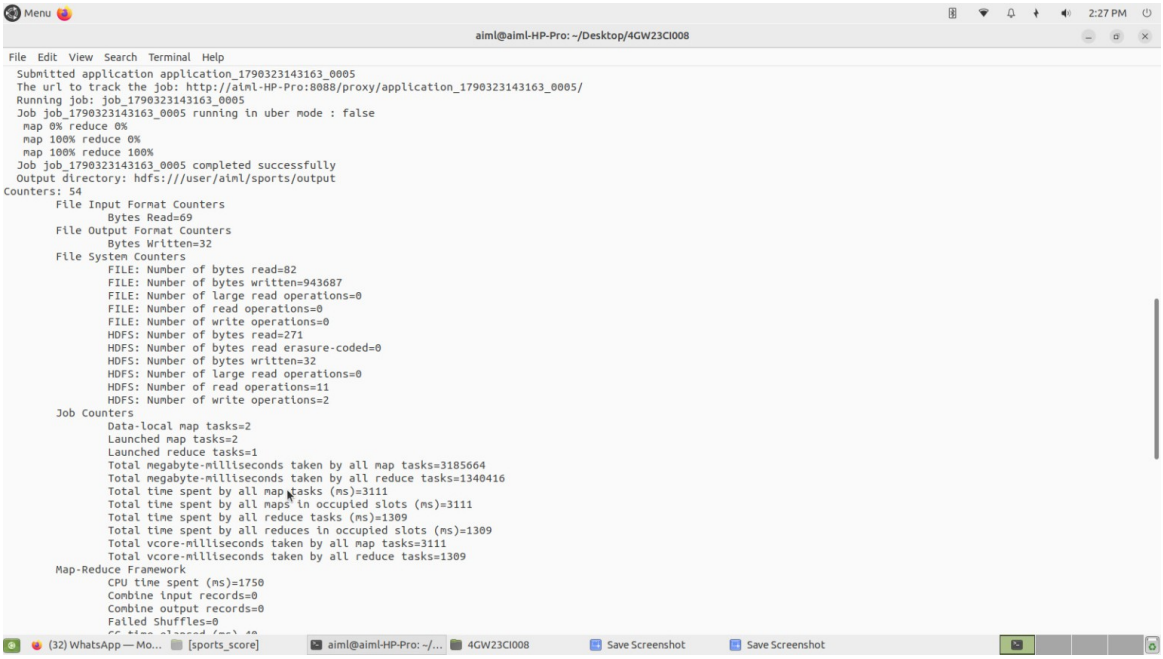


Figure 3. Hadoop MapReduce counters showing Map input/output records and Reduce groups/output.

The two Reduce groups correspond to the two players: Rohit and Virat.

9. MapReduce Output

After the job completes, the output directory contains _SUCCESS and part-00000. The final output is displayed using the HDFS cat command.

```
hdfs dfs -ls /user/$USER/sports/output
hdfs dfs -cat /user/$USER/sports/output/part-00000
```



Figure 4. HDFS output directory and final MapReduce results: Rohit 75.00 and Virat 90.00.

The Hadoop output was then merged into a local file using hdfs dfs -getmerge.

```
hdfs dfs -getmerge /user/$USER/sports/output sports_output.txt
cat sports_output.txt
```

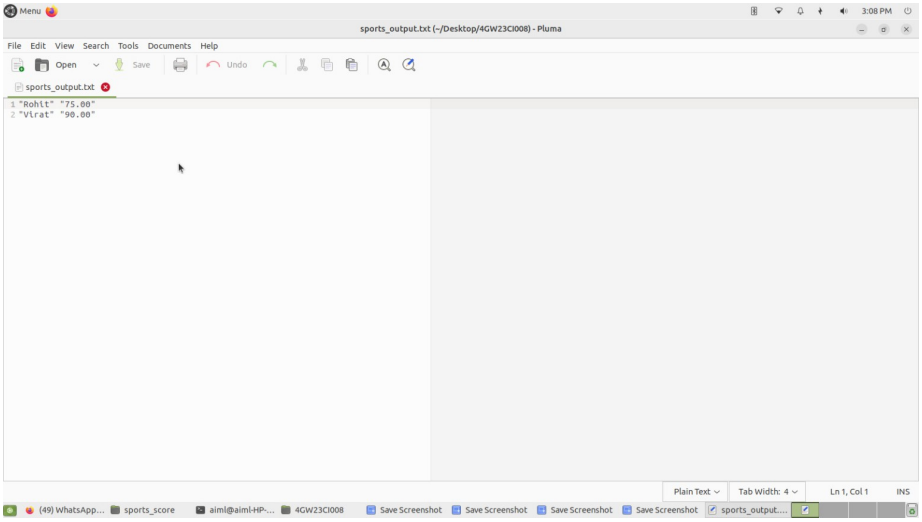


Figure 5. Local sports_output.txt displaying the final output.

10. Sports Score Calculation

The five input records are grouped by player and the average is calculated as follows:

Player	Scores	Total	Count	Average Score
Virat	80 + 90 + 100	270	3	90.00
Rohit	70 + 80	150	2	75.00

11. Brief Discussion

Map stage: Every record is read and split into two fields: player name and score. The Mapper generates a (player, score) pair for every input record.

Shuffle and Sort: Hadoop automatically groups all scores belonging to the same player. For example, Virat → [80, 90, 100] and Rohit → [70, 80].

Reduce stage: The Reducer calculates the total and count of scores for each player and divides the total by the count to obtain the average.

12. Discussion Questions

1. Why is HDFS used?
HDFS stores the sports score data as the distributed input dataset for the MapReduce job.

2. Why is the player name used as the Mapper key?
The requirement is to calculate the average score separately for each player, so the player name provides the grouping key.

3. What does the Reducer do?
It receives all score values for the same player and calculates their total, count, and average.

4. What is the role of YARN?
YARN manages Hadoop resources and supports execution of the MapReduce job.

5. What does mrjob provide?
mrjob allows the MapReduce logic to be written in Python and executed using a Hadoop runner against the HDFS input.

13. Result

The Hadoop MapReduce job was successfully executed using Python and the mrjob library. The supplied terminal evidence shows successful completion and the final output was stored in HDFS and in the local file sports_output.txt.

Player	Average Score
Virat	90.00
Rohit	75.00

14. Conclusion

The Hadoop MapReduce program was successfully implemented using Python and the mrjob library on a single-node Hadoop cluster running on Ubuntu. The sports score data was stored in HDFS and processed using MapReduce on YARN.

The Mapper successfully extracted the player name and score from each input record. Hadoop grouped the scores belonging to the same player, and the Reducer calculated the average score for each player.

The program correctly produced the average scores: Virat → 90.00 and Rohit → 75.00. Thus, the objective of calculating the average score for each player using Apache Hadoop MapReduce and Python mrjob was successfully achieved.